ICS 202 – Data Structures and Algorithms

Topic 11: Hashing

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Conventions (as in the slides): open-addressing tables are listed by index $0, 1, \ldots, n-1$, with – for an EMPTY cell. Probe sequences are $h_i(key) = [h(key) + c(i)] \,\%\, n$; quadratic probing with $c(i) = \pm i^2$ tries $+1, -1, +4, -4, \ldots$ ($+i^2$ before $-i^2$), and a negative index is normalised as $(index + n) \,\%\, n$. Deletion is lazy (the cell is marked DELETED). An insertion keeps probing past DELETED cells until it reaches an EMPTY cell (checking for a duplicate), then stores the key in the first DELETED cell it passed, or in the EMPTY cell if it passed none.

Part A: Multiple choice

Q1.

Activity 11-01 (T252)

How many storage cells would be wasted in an array-based implementation with $O(1)$ access (the ID used directly as the index) for 10,000 student records, each identified by a 7-digit ID number?

(A) 990,000

(B) 9,990,000

(C) 10,000,000

(D) 999,990,000

Answer: (B). The array needs 10^7 cells for all 7-digit IDs, and only 10^4 are used: $10^7 - 10^4 = 9{,}990{,}000$.

My answer

Q2.

Quiz 4 (T242)

The main objective of using hashing is to

(A) attempt achieving $O(\log n)$ search cost.

(B) have most efficient space complexity.

(C) attempt achieving $\Theta(1)$ search cost.

(D) None of the answers given is correct.

Answer: (C). Hashing jumps straight to the place where the key should be, so search cost does not depend on n.

My answer

Q3. Quiz 4 (T242)

Consider development of a suitable hash function to store student records, using the KFUPM ID, consisting of 9 digits as follows: $y_1y_2y_3y_4z_1z_2z_3z_4\,0$, where $y_1y_2y_3y_4$ is the year of joining KFUPM, z_i could be any digit between 0 and 9 for $i \in \{1, 2, 3, 4\}$, and the last digit is zero. Which of the following hash functions is most suitable?

(A) $h(ID) = ID/100000$ (integer division).

(B) $h(ID) = (ID \,\%\, 100000)/10$.

(C) $h(ID) = y_2y_3y_4z_4$.

(D) $h(ID) = z_2z_3z_4\,0$.

Answer: (B). It extracts $z_1z_2z_3z_4$, the four digits that really vary; (A) keeps only the year (huge collisions), (C) mostly uses the year, (D) drops z_1 and keeps the constant 0.

My answer

Q4. Hashing Old Exams

The most efficient data structure in the worst case, among the following, in the cost of searching is

(A) an open-addressing hash table.

(B) a binary search tree.

(C) a sorted array.

(D) both answers (B) and (C) as they are asymptotically equal.

Answer: (C). Binary search on a sorted array is $O(\log n)$ in the worst case; a hash table (all keys colliding) and a BST (degenerate) are $O(n)$.

My answer

Q5. Quiz 4 (T242)

When choosing a table size for a hash table using separate chaining with $h(key) = key \,\%\, tableSize$, collisions are reduced if we choose it to be

(A) a power of 2.

(B) any suitable number greater than the number of keys.

(C) a power of 10.

(D) a prime number not close to a power of 2.

Answer: (D). With the division-remainder method the table size is better if it is prime; a power of 2 or 10 just keeps the last bits/digits of the key.

My answer

Q6. Quiz 4 (T242)

Consider the following separate chaining hash table with $h(key) = key \,\%\, 7$ (lists shown from head to tail): index 0: $21 \to 14 \to 7$; index 1: $8 \to 15$; index 2: 23; index 6: 13; indexes 3, 4, 5: empty lists. The load factor of this hash table is

(A) $4/7$.

(B) $7/6$.

(C) $2/3$.

(D) 1.

Answer: (D). $\lambda =$ number of items/table size $= 7/7 = 1$.

My answer

Q7. Quiz 4 (T242)

For the same separate chaining table as in the previous question ($h(key) = key \,\%\, 7$), the key 41 would have been inserted into the linked list of index

(A) 6

(B) 5

(C) 4

(D) 3

Answer: (A). $41 \,\%\, 7 = 6$.

My answer

Q8. Activity 11-01 (T252)

Insert the keys 15, 11, 25, 16, 9, 8, and 12, in that order, into a hash table of size 7 using separate chaining with the hash function $h(key) = key \bmod 7$. Which describes the resulting table (each list in insertion order; unlisted indexes are empty)?

(A) index 1: 15, 8; index 2: 16, 9; index 3: 11, 25; index 5: 12

(B) index 1: 15, 8; index 2: 16, 9; index 4: 11, 25; index 6: 12

(C) index 1: 15, 8; index 2: 16, 9; index 4: 11, 25; index 5: 12

(D) index 1: 15, 8, 12; index 2: 16, 9; index 4: 11, 25

Answer: (C). $15, 8 \to 1$; $16, 9 \to 2$; $11, 25 \to 4$; $12 \to 5$.

My answer

Q9. Hashing Old Exams

Given a hash table of size 7 and hash function $h(key) = key \,\%\, 7$, the operations insert(14), insert(2), insert(42), insert(8), insert(21) are performed using separate chaining. Which describes the final table (lists in insertion order)?

(A) list 0: 14, 42; list 1: 21, 8; list 2: 2

(B) list 0: 14, 42, 21; list 1: 8; list 2: 2

(C) list 0: 14, 21; list 1: 42, 8; list 2: 2

(D) list 0: 14; list 1: 42, 8; list 2: 2; list 6: 21

Answer: (B). $14, 42, 21$ all hash to 0, $8 \to 1$, $2 \to 2$; chaining never moves a key to another index.

My answer

Q10. Quiz 4 (T241)

Which of the following statements is true regarding separate chaining?

(A) Duplicate keys are not allowed in separate chaining.

(B) The size of the hash table must be a prime number.

(C) Deletion of a key in separate chaining costs $\Theta(1)$ in the worst case.

(D) The load factor in separate chaining can exceed 1, but it may not negatively affect the performance.

Answer: (D). With chaining λ often exceeds 1 and performance declines slowly; keys need not be unique, size need not be prime, and deletion must search a list ($O(n)$ worst case).

My answer

Q11. Quiz 5 (T251)

Which statement correctly compares separate chaining and open addressing when the load factor is greater than 1?

(A) Both methods fail, as we cannot store more keys than the number of available cell indexes.

(B) Open addressing can store all keys as long as the load factor is less than 2.

(C) Separate chaining works normally, but open addressing must resize, otherwise it will fail.

(D) Both methods perform equally well, but with search costs.

Answer: (C). In open addressing every key occupies a cell, so $\lambda \leq 1$; chaining stores extra keys in the lists.

My answer

Q12. Activity 11-01 (T252)

The keys 12, 18, 13, 2, 3, 23, 5, and 15 are inserted into an initially empty hash table of length 10 using open addressing with the hash function $h(k) = k \bmod 10$ and linear probing. What is the resulting hash table (indexes 0 to 9)?

(A) 15, –, 12, 13, 2, 3, 23, 5, 18, –

(B) –, –, 12, 2, 13, 3, 23, 5, 18, 15

(C) –, –, 12, 13, 2, 23, 3, 5, 18, 15

(D) –, –, 12, 13, 2, 3, 23, 5, 18, 15

Answer: (D). 2: 2,3,4; 3: 3,4,5; 23: 3,4,5,6; 5: 5,6,7; 15: 5,6,7,8,9.

My answer

Q13. Assignment 3 (T252)

The keys 18, 41, 22, 44, 59, 32, 31, 73, 85, 26, 95 are inserted, in this order, into an empty hash table of size 13 with $h(k) = k \bmod 13$ using open addressing with linear probing. The final indexes of 31, 73 and 85 are, respectively,

(A) 10, 11, 12

(B) 5, 8, 7

(C) 4, 8, 11

(D) 11, 8, 10

Answer: (A). The primary cluster 5–9 grows: 31: 5,6,7,8,9,10; 73: 8,9,10,11; 85: 7,8,9,10,11,12.

My answer

Q14. Hashing Old Exams

What is primary clustering, and how does double hashing avoid it?

(A) Keys with the same home address follow the same probe sequence; double hashing avoids it by using $c(i) = i^2$.

(B) Many keys are stored in the same linked list in separate chaining; double hashing avoids it by using a prime table size.

(C) Long runs of consecutive occupied cells form, since any key hashing into a run probes to its end and makes it longer (runs also merge); double hashing avoids it because the step $h_p(key)$ depends on the key, so keys landing in a run jump away by different step sizes.

(D) Many DELETED cells accumulate after deletions; double hashing avoids it by rehashing the table.

Answer: (C). Primary clustering is the linear-probing effect; with $c(i) = i \cdot h_p(key)$ keys do not follow the same consecutive cells.

My answer

Q15. Hashing Old Exams

Which of the following statements is true regarding secondary clustering?

(A) Adjacent smaller clusters combine to form bigger clusters.

(B) If $h(k_1) = a$ and $h(k_2) = b$ where $a \neq b$, and both belong to the same cluster of keys, they will have the same probe sequence.

(C) If $h(k_1) = a$ and $h(k_2) = a$, they will have the same probe sequence.

(D) Statements in (A) and (B) are both true.

Answer: (C). Secondary clustering (quadratic probing): synonyms follow exactly the same probe sequence. (A) describes primary clustering.

My answer

Q16. Hashing Old Exams

With respect to the hash function $h(k) = k \,\%\, tableSize$ in open addressing (quadratic probing), the $tableSize$ value that is most suitable is

(A) any prime number of the form $4j + 3$ where j is any positive integer.

(B) any even number.

(C) any odd number.

(D) any number of the form 2^p, where p is a prime number.

Answer: (A). A prime of the form $4j + 3$ guarantees that $\pm i^2$ probing reaches every cell (Property 2).

My answer

Q17. Hashing Old Exams

Given a hash table of size 7 and hash function $h(key) = key \,\%\, 7$, show the final state after insert(14), insert(2), insert(42), insert(8), insert(21) using open addressing with $c(i) = \pm i^2$ (indexes 0 to 6).

(A) 14, 42, 2, –, 21, 8, –

(B) 14, 42, 2, –, –, 8, 21

(C) 14, 42, 2, 8, 21, –, –

(D) 14, 42, 2, –, 8, 21, –

Answer: (B). 42: 0,1; 8: 1,2,0,5; 21: 0,1, then $(0 - 1 + 7) \,\%\, 7 = 6$.

My answer

Q18. Activity 11-01 (T252)

Insert the keys 23, 13, 21, 14, 7, 8, and 15 in that order into a hash table of size 7 using quadratic probing with $c(i) = \pm i^2$ and the hash function $h(key) = key \,\%\, 7$. How many collisions occur during the insertion process?

(A) 8

(B) 10

(C) 12

(D) 14

Answer: (C). 14: 1 collision (0); 7: 3 (0,1,6); 8: 3 (1,2,0); 15: 5 (1,2,0,5,4); total 12.

My answer

Q19. Quiz 4 (T232)

The following hash table was filled using $h(key) = (key + 5) \,\%\, 13$ and quadratic probing $c(i) = \pm i^2$: 0: -5 (O), 1: 13 (O), 2: 30 (O), 3: E, 4: E, 5: 0 (DELETED), 6: 26 (O), 7: 15 (O), 8: E, 9: E, 10: E, 11: 6 (O), 12: 16 (O). What is the complete probe sequence of inserting the key 1, and where is it stored?

(A) 6, 7, 5; stored at index 5

(B) 6, 7, 5, 10; stored at index 5

(C) 6, 7, 5, 10; stored at index 10

(D) 6, 7, 8; stored at index 8

Answer: (B). $h(1) = 6$ (O), $6 + 1 = 7$ (O), $6 - 1 = 5$ (DELETED, remember it), $6 + 4 = 10$ (EMPTY, stop: 1 is not a duplicate); 1 goes into the DELETED cell 5.

My answer

Q20. Quiz 4 (T232)

For the hash table of the previous question, what is the load factor after inserting key 1?

(A) $6/13$

(B) $7/13$

(C) $8/13$

(D) $9/13$

Answer: (C). 7 occupied cells before (the DELETED cell does not count) plus key 1 gives $8/13$.

My answer

Q21. Quiz 4 (T241)

Consider the following hash table of size 13, where keys were inserted using $h(k) = k \,\%\, 13$ and quadratic probing with $c(i) = \pm i^2$: 0: 1 (O), 1: 14 (O), 2: 27 (O), 3: E, 4: 17 (O), 5: 40 (O), 6–9: E, 10: 24 (D), 11: 11 (O), 12: 12 (O). The number of hash table entries inspected before inserting key 37, and the index where 37 is inserted, are

(A) 3 entries; index 10

(B) 5 entries; index 10

(C) 5 entries; index 7

(D) 6 entries; index 8

Answer: (B). $h(37) = 11$; probes 11, 12, 10 (D), $11 + 4 = 15 \,\%\, 13 = 2$, $11 - 4 = 7$ (E): 5 entries; 37 is placed in the DELETED cell 10.

My answer

Q22. Quiz 5 (T242)

Consider the following hash table of size 11, where keys were inserted using $h(k) = k \,\%\, 11$ and quadratic probing with $c(i) = \pm i^2$ ($+i^2$ is used before $-i^2$): 0: 22 (O), 1: 34 (O), 2: E, 3: E, 4: 15 (O), 5: E, 6: 17 (O), 7: 18 (D), 8: E, 9: 9 (O), 10: 0 (O). The number of hash table indexes inspected before inserting key 11, and the index where 11 is inserted, are

(A) 5 indexes; index 8

(B) 6 indexes; index 5

(C) 7 indexes; index 7

(D) 3 indexes; index 3

Answer: (C). Probes 0, 1, 10, 4, 7 (D), 9, 2 (E): 7 indexes; 11 goes into the first DELETED cell, index 7.

My answer

Q23. Quiz 5 (T251)

The following hash table was filled using $h(key) = (key+5) \,\%\, 11$ and quadratic probing with $c(i) = \pm i^2$: 0: 17, 1: 18, 2: 30, 3: 20, 4: 10, 5: 22, 6–9: EMPTY, 10: 5 (all non-empty cells OCCUPIED). What is the complete probe sequence of inserting the key 29?

(A) 1, 2, 0, 5, 8; stored at index 8

(B) 1, 2, 3, 4, 5, 6; stored at index 6

(C) 1, 2, 5, 10, 6; stored at index 6

(D) 1, 2, 0, 5, 8, 9; stored at index 9

Answer: (A). $h(29) = 1$; $1 + 1 = 2$, $1 - 1 = 0$, $1 + 4 = 5$, $1 - 4 = -3 \to 8$ (EMPTY).

My answer

Q24. Hashing Old Exams

The keys 33, 34, 31, 20, 21, 22, 44, 41, 8 are inserted, in this order, into a hash table of size 11 using open addressing, $h(key) = key \,\%\, 11$ and quadratic probing with $c(i) = \pm i^2$. The final table (indexes 0 to 10) is

(A) 33, 34, –, 21, 22, 44, 8, –, 41, 31, 20

(B) 33, 34, 21, 22, 44, 8, –, –, 41, 31, 20

(C) 33, 34, –, 21, 22, –, 8, 44, 41, 31, 20

(D) 33, 34, –, 21, 8, 44, 22, –, 41, 31, 20

Answer: (C). 20: 9,10; 21: 10,0,9,3; 22: 0,1,10,4; 44: 0,1,10,4,7; 8: 8,9,7,1,4,6.

My answer

Q25. Hashing Old Exams

A hash table of size 11 with $h(x) = x \bmod 11$ already holds 33 at index 0 and 21 at index 10. The values 28, 17, 39, 50 are then loaded, in this order, using $c(i) = \pm i^2$. The final indexes of 28, 17, 39 and 50 are, respectively,

(A) 6, 7, 5, 2

(B) 6, 7, 4, 9

(C) 6, 3, 5, 7

(D) 6, 7, 8, 9

Answer: (A). All hash to 6. 17: 6,7; 39: 6,7,5; 50: 6,7,5,10,2.

My answer

Q26. Quiz 5 (T251)

Suppose a hash table of size $n = 11$ uses double hashing with $h(k) = k \bmod 11$ and $h_p(k) = 1 + (k \bmod 10)$. Inserting key $k = 32$, which of the following gives the correct probe sequence?

(A) 10, 0, 1, 2, ...

(B) 10, 1, 3, 6, ...

(C) 10, 2, 7, 8, ...

(D) 10, 2, 5, 8, ...

Answer: (D). $h(32) = 10$, $h_p(32) = 3$: $(10 + 3i)\,\%\,11 = 10, 2, 5, 8, \ldots$

My answer

Q27.

Hashing Old Exams

Given a hash table of size 7 and $h(key) = key\,\%\,7$, show the final state after insert(14), insert(2), insert(42), insert(8), insert(21) using open addressing with $c(i) = i \cdot h_p(x)$, where $h_p(x) = 1 + x \bmod 6$ (indexes 0 to 6).

(A) 14, 42, 2, –, 8, 21, –

(B) 14, 42, 2, –, 21, 8, –

(C) 14, 42, 2, –, –, 8, 21

(D) 14, 42, 2, 8, 21, –, –

Answer: (A). 42: $h_p = 1$, 0,1; 8: $h_p = 3$, 1,4; 21: $h_p = 4$, 0,4,1,5.

My answer

Q28.

Hashing Old Exams

The keys 16, 35, 33, 38, 18, 27, 10 are inserted, in this order, into a hash table of size 17 using open addressing, $h(key) = key\,\%\,17$ and double hashing with $h_p(key) = 11 - key\,\%\,11$. The occupied cells of the final table (index:key) are

(A) 0:33, 1:35, 2:18, 4:38, 10:27, 11:10, 16:16

(B) 1:35, 4:38, 5:18, 10:33, 11:27, 12:10, 16:16

(C) 1:35, 3:33, 4:38, 7:18, 10:27, 15:10, 16:16

(D) 1:35, 4:38, 5:18, 10:33, 12:27, 11:10, 16:16

Answer: (B). 33: $h_p = 11$, 16,10; 18: $h_p = 4$, 1,5; 27: $h_p = 6$, 10,16,5,11; 10: $h_p = 1$, 10,11,12.

My answer

Q29.

Quiz 4 (T241)

Consider inserting keys 16 and 33, respectively, into an initially empty hash table of size 17, using open addressing with hash function $h(key) = key\,\%\,17$ and double hashing for collision resolution, with $h_p(key) = 11 - (key\,\%\,11)$. Key 33 will be inserted into index

(A) 15

(B) 14

(C) 10

(D) 8

Answer: (C). $h(33) = 16$ collides with 16; $h_p(33) = 11$; $(16 + 11)\,\%\,17 = 10$.

My answer

Q30.

Hashing Old Exams

The keys 33, 34, 31, 20, 21, 22, 44, 41, 8 are inserted, in this order, into a hash table of size 11 using open addressing, $h(key) = key\,\%\,11$ and double hashing with $h_p(key) = 1 + key\,\%\,10$. The final table (indexes 0 to 10) is

(A) 33, 34, –, 21, 22, –, 8, 44, 41, 31, 20

(B) 33, 34, 21, 22, 44, 8, –, –, 41, 31, 20

(C) 33, 34, –, 21, 8, 44, 22, –, 41, 31, 20

(D) 33, 34, –, 21, 22, 44, 8, –, 41, 31, 20

Answer: (C). 20: $h_p = 1$, 9,10; 21: $h_p = 2$, 10,1,3; 22: $h_p = 3$, 0,3,6; 44: $h_p = 5$, 0,5; 8: $h_p = 9$, 8,6,4.

My answer

Q31.

Hashing Old Exams

A hash table of size 11 with $h(x) = x \bmod 11$ already holds 33 at index 0 and 21 at index 10. The values 28, 17, 39, 50 are then loaded, in this order, using $c(i) = i \cdot h_p(x)$ where $h_p(x) = 1 + x \bmod 10$. The final indexes of 28, 17, 39 and 50 are, respectively,

(A) 6, 7, 5, 2

(B) 6, 3, 5, 7

(C) 6, 7, 8, 9

(D) 6, 3, 7, 5

Answer: (B). 17: $h_p = 8$, 6,3; 39: $h_p = 10$, 6,5; 50: $h_p = 1$, 6,7.

My answer

Q32.

Activity 11-01 (T252)

Insert the keys 18, 26, 35, 9, 64, 47, 96, 36, 70, 12 in that order into an empty hash table of size 13 using double hashing with $h(key) = key\,\%\,13$ and $h_p(key) = 1 + key\,\%\,12$. Then perform find(7). Which is correct?

(A) 70 is at index 3 and 12 at index 1; find(7) probes 7 then 2 and fails because cell 2 is EMPTY.

(B) 70 is at index 3 and 12 at index 1; find(7) probes 7, 8, 9, 10, 11, 12, 0, 1, 2 and fails.

(C) 70 is at index 7 and 12 at index 1; find(7) finds 7 is not present after probing only cell 7.

(D) 70 is at index 3 and 12 at index 0; find(7) probes 7 then 2 and fails because cell 2 is EMPTY.

Answer: (A). 70: $h_p = 11$, 5,3; 12: $h_p = 1$, 12,0,1; find(7): $h_p = 8$, cell 7 holds 96, $(7 + 8)\,\%\,13 = 2$ is EMPTY.

My answer

Q33.

Quiz 4 (T251)

A hash table of size 7 holds 15 at index 1, 24 at index 3, 4 at index 4 and 38 at index 6 (all OCCUPIED; the other cells EMPTY). Using double hashing with $h(key) = key\,\%\,7$ and $h_p(key) = 1 + (key\,\%\,6)$, the operations Insert(20), Delete(24), Insert(25) are performed. The final table (indexes 0 to 6; D = DELETED) is

(A) –, 15, 20, D, 4, 25, 38

(B) 20, 15, –, 25, 4, –, 38

(C) –, 15, 20, 25, 4, –, 38

(D) –, 15, 20, 24, 4, 25, 38

Answer: (C). 20: $h_p = 3$, 6,2. Delete 24 marks cell 3 DELETED. 25: $h_p = 2$, 4,6,1,3 (D),5 (E): 25 goes into DELETED cell 3.

My answer

Q34.

Assignment 3 (T252)

The keys 18, 41, 22, 44, 59, 32, 31, 73, 85, 26, 95 are inserted, in this order, into an empty hash table of size 13 with $h(k) = k \bmod 13$ using double hashing with $h_p(key) = 11 - (key \bmod 11)$. The final indexes of 44, 31 and 85 are, respectively,

(A) 6, 10, 12

(B) 3, 11, 10

(C) 6, 4, 11

(D) 3, 10, 11

Answer: (B). 44: $h_p = 11$, 5,3; 31: $h_p = 2$, 5,7,9,11; 85: $h_p = 3$, 7,10.

My answer

Q35.

Hashing Old Exams

Which of the following statements is incorrect regarding hashing?

(A) Load factors greater than 1 in separate chaining may not negatively affect the performance of the hash table.

(B) In open addressing, the probing sequence $c(i) = i^2$ guarantees that any hash table of size equal a prime number can reach a load factor of 100%.

(C) Linear probing suffers from the primary clustering problem.

(D) Double hashing results in a better search performance than quadratic probing.

Answer: (B). Quadratic probing tries only about half of the cells ($i \leq (n-1)/2$), so it can fail to find an empty cell before the table is full.

My answer

Q36.

Quiz 4 (T241)

For the following hash table of size 11, its load factor is equal to
0: 7 (O), 1: 13 (D), 2: 11 (O), 3: 11 (D), 4: 17 (O), 5: 40 (O), 6: 60 (O), 7: E, 8: E, 9: E, 10: 9 (D).

(A) $5/11$

(B) $8/11$

(C) $6/11$

(D) $3/11$

Answer: (A). Only OCCUPIED cells hold items: 7, 11, 17, 40, 60, so $\lambda = 5/11$.

My answer

Q37.

Activity 11-01 (T252)

Can two hash tables with the same load factor perform differently?

(A) No; performance depends only on the load factor.

(B) No, as long as both tables have the same size.

(C) Yes; performance also depends on the collision resolution technique and on how evenly the hash function spreads the keys (clustering), so the number of probes can differ.

(D) Yes, but only when one table is larger than the other.

Answer: (C). Same λ but, e.g., linear probing with clustered keys versus double hashing with well-spread keys give very different probe counts.

My answer

Q38.

Activity 11-01 (T252)

A hash table of size 23 uses linear probing and $h(key) = key \,\%\, 23$. Its cells are: 1, 2: DELETED; 6: DELETED; 7: 29; 8, 9, 10, 11: DELETED; 12: 54; 13: DELETED; 14: 60; 15, 16: DELETED; 17: 35; 18, 19: DELETED; 21: 21; 22: DELETED; all other cells EMPTY. How many probes are required to search for 75?

(A) 2

(B) 14

(C) 15

(D) 16

Answer: (C). $h(75) = 6$; cells 6 to 19 are DELETED or hold other keys, cell 20 is EMPTY: 15 probes (only 2 after rehashing away the DELETED cells).

My answer

Q39.

Activity 11-01 (T252)

If rehashing doubles the table size, how does the load factor change immediately after resizing?

(A) It doubles.

(B) It stays the same, since the same keys are stored.

(C) It becomes approximately half of its previous value.

(D) It becomes 1.

Answer: (C). Same number of items, about twice the table size, so λ is roughly halved.

My answer

Q40.

Quiz 5 (T251)

With open addressing, which condition ensures that a search for a key k must terminate without finding the key?

(A) Encountering a slot containing a different key with the same hash value.

(B) Encountering an EMPTY slot.

(C) Encountering a DELETED slot.

(D) Making exactly $\lfloor n/2 \rfloor$ probes without success, where n is the table size.

Answer: (B). A DELETED slot or a different key means continue; an EMPTY slot means the key was never placed further along the sequence.

My answer

Part B: Programming

Q41. [new]

Write the method `public static int hash(String key, int tableSize)` that returns the character-sum hash $h = (c_0 + c_1 + \cdots + c_{n-1}) \,\%\, tableSize$ of a string key (each c_i is the character code). For example, `hash("salt", 13)` returns $436 \,\%\, 13 = 7$.

Answer:

```
public static int hash(String key, int tableSize) {
    int hashValue = 0;
    for (int i = 0; i < key.length(); i++)
        hashValue += key.charAt(i);
    return hashValue % tableSize;
}
```

My answer

Q42. [new]

Write `public static int polyHash(String s, int p, int tableSize)` that returns the polynomial hash $h = \left(\sum_{k=0}^{s.length()-1} c_k \cdot p^k\right) \,\%\, tableSize$, where c_k is the code of `s.charAt(k)`. Avoid overflow by applying `%` at every step (use Horner's rule starting from the last character).

Answer:

```
public static int polyHash(String s, int p, int tableSize) {
    long h = 0;
    for (int k = s.length() - 1; k >= 0; k--)
        h = (h * p + s.charAt(k)) % tableSize;
    return (int) h;
}
```

My answer

Q43. [new]

A separate chaining hash table of integers is declared as below, with $h(key) = key \,\%\, table.length$.

Complete `insert` (add at the end of the list), `search` and `delete` (return `false` if the key is absent). Keep `size` up to date.

```
public class ChainedHashTable {
    private LinkedList<Integer>[] table;
    private int size;   // number of keys stored
    public ChainedHashTable(int n) {
        table = new LinkedList[n];
        for (int i = 0; i < n; i++) table[i] = new LinkedList<>();
    }
    private int h(int key) { return key % table.length; }
    public void insert(int key)    { /* ... */ }
    public boolean search(int key) { /* ... */ }
    public boolean delete(int key) { /* ... */ }
}
```

Answer:

```
public void insert(int key) {
    table[h(key)].addLast(key);   // synonyms chained in the same list
    size++;
}
public boolean search(int key) {
    return table[h(key)].contains(key);
}
public boolean delete(int key) {
    boolean removed = table[h(key)].remove(Integer.valueOf(key));
    if (removed) size--;
    return removed;
}
```

My answer

Q44. [new]

Add two methods to the `ChainedHashTable` class of the previous question: `public double loadFactor()` returning $\lambda = size/table.length$, and `public int longestChain()` returning the length of the longest linked list. For keys 23, 13, 21, 14, 7, 8, 15 in a table of size 7, `longestChain()` returns 3 and `loadFactor()` returns 1.0.

Answer:

```
public double loadFactor() {
    return (double) size / table.length;
}
public int longestChain() {
```

```
    int max = 0;
    for (LinkedList<Integer> list : table)
        max = Math.max(max, list.size());
    return max;
}
```

My answer

Q45. [new]

An open-addressing hash table with linear probing is declared as below ($h(key) = key \,\%\, n$, $c(i) = i$). Write `public int find(int key)` that returns the index of `key`, or -1 if it is not present. The search continues past DELETED cells and cells holding other keys, and stops at an EMPTY cell or after n probes.

```
public class LinearProbingHashTable {
    static final int EMPTY = 0, OCCUPIED = 1, DELETED = 2;
    int[] keys;      // keys[j] is meaningful when status[j] != EMPTY
    int[] status;    // all EMPTY initially
    int count;       // number of OCCUPIED cells
    public LinearProbingHashTable(int n) { keys = new int[n]; status = new int[n]; }
    int h(int key) { return key % keys.length; }
}
```

Answer:

```
public int find(int key) {
    int n = keys.length;
    for (int i = 0; i < n; i++) {
        int j = (h(key) + i) % n;
        if (status[j] == EMPTY) return -1;            // search fails
        if (status[j] == OCCUPIED && keys[j] == key) return j;
        // DELETED or a different key: continue
    }
    return -1;
}
```

My answer

Q46. [new]

For the `LinearProbingHashTable` of the previous question, write `public boolean insert(int key)` following the slides: probe $h_0, h_1, \ldots$; if the key is found in an OCCUPIED cell it is a duplicate (return `false`); remember the first DELETED cell passed; when an EMPTY cell is reached, store the key in the first DELETED cell passed (if any), otherwise in the EMPTY cell. If no EMPTY cell exists, use the first DELETED cell, or return `false` if the table is full.

Answer:

```
public boolean insert(int key) {
    int n = keys.length, firstDeleted = -1;
    for (int i = 0; i < n; i++) {
        int j = (h(key) + i) % n;
        if (status[j] == OCCUPIED && keys[j] == key) return false; // duplicate
        if (status[j] == DELETED && firstDeleted == -1) firstDeleted = j;
        if (status[j] == EMPTY) {
            int target = (firstDeleted != -1) ? firstDeleted : j;
            keys[target] = key; status[target] = OCCUPIED; count++;
            return true;
        }
    }
    if (firstDeleted != -1) {
        keys[firstDeleted] = key; status[firstDeleted] = OCCUPIED; count++;
        return true;
    }
    return false;   // table full
}
```

My answer

Q47. [new]

For the `LinearProbingHashTable` above, write `public boolean delete(int key)` using lazy deletion: locate the key with `find`; if found, mark its cell DELETED (not EMPTY) and decrement `count`; return whether the deletion succeeded. Explain in a comment why the cell is not set to EMPTY.

Answer:

```
public boolean delete(int key) {
    int j = find(key);
    if (j == -1) return false;      // deletion fails
    // DELETED, not EMPTY: otherwise searches for keys that probed
    // past this cell would stop here and fail.
    status[j] = DELETED;
    count--;
    return true;
}
```

My answer

Q48. [new]

For quadratic probing with $c(i) = \pm i^2$ in a table of size n (odd), write `public int[] probeSequence(int key)` that returns the n indexes $h_0, h_{+1}, h_{-1}, h_{+2}, h_{-2}, \ldots$ for $i = 1, \ldots, (n-1)/2$, where $h(key) = key \,\%\, n$ and negative indexes are normalised as $(index + n) \,\%\, n$. Assume the class has `int[] keys` and `int h(int key)`. For $n = 7$ and key 15 the result is `[1, 2, 0, 5, 4, 3, 6]`.

Answer:

```
public int[] probeSequence(int key) {
    int n = keys.length;
```

```
    int[] seq = new int[n];
    seq[0] = h(key);
    int k = 1;
    for (int i = 1; i <= (n - 1) / 2; i++) {
        seq[k++] = (h(key) + i * i) % n;
        seq[k++] = ((h(key) - i * i) % n + n) % n;   // normalise
    }
    return seq;
}
```

My answer

Q49. [new]

A double-hashing table has the same fields as `LinearProbingHashTable` (`keys`, `status`, constants EMPTY/OCCUPIED/DELETED), with $h(key) = key \,\%\, n$ and $h_p(key) = 1 + key \,\%\, (n-1)$. Write `int hp(int key)` and `public int insert(int key)` using $h_i(key) = [h(key) + i \cdot h_p(key)] \,\%\, n$; it returns the index used, or -1 for a duplicate or a full table (same DELETED-cell rule as the slides). For $n = 13$ and keys 18, 26, 35, 9, 64, 47, 96, 36, 70, 12 the returned indexes are 5, 0, 9, 6, 12, 8, 7, 10, 3, 1.

Answer:

```
int hp(int key) { return 1 + key % (keys.length - 1); }

public int insert(int key) {
    int n = keys.length, firstDeleted = -1;
    for (int i = 0; i < n; i++) {
        int j = (h(key) + i * hp(key)) % n;
        if (status[j] == OCCUPIED && keys[j] == key) return -1;  // duplicate
        if (status[j] == DELETED && firstDeleted == -1) firstDeleted = j;
        if (status[j] == EMPTY) {
            int t = (firstDeleted != -1) ? firstDeleted : j;
            keys[t] = key; status[t] = OCCUPIED;
            return t;
        }
    }
    if (firstDeleted != -1) {
        keys[firstDeleted] = key; status[firstDeleted] = OCCUPIED;
        return firstDeleted;
    }
    return -1;
}
```

My answer

Q50. [new]

Write `public void rehash()` for `LinearProbingHashTable`: create a new table whose size is the smallest prime $\geq 2\times$ the old size, scan the old table from index 0 and insert every OCCUPIED key into the new table with the new hash function (DELETED cells are dropped). You may use `insert` from the earlier question and write a helper `isPrime`. For example, a size-7 table holding 13, 15, 23, 24, 6 (at indexes 0, 1, 2, 3, 6) becomes a size-17 table with 23 at 6, 24 at 7, 6 at 8, 13 at 13, 15 at 15.

Answer:

```
public void rehash() {
    int[] oldKeys = keys, oldStatus = status;
    int newSize = 2 * oldKeys.length;
    while (!isPrime(newSize)) newSize++;
    keys = new int[newSize];
    status = new int[newSize];      // all EMPTY
    count = 0;
    for (int i = 0; i < oldKeys.length; i++)
        if (oldStatus[i] == OCCUPIED)
            insert(oldKeys[i]);     // h(key) now uses newSize
}
static boolean isPrime(int x) {
    if (x < 2) return false;
    for (int d = 2; d * d <= x; d++)
        if (x % d == 0) return false;
    return true;
}
```

My answer